```
/* backend/db.js */
const mysql = require('mysql2');

const db = mysql.createConnection({
    host: 'localhost',
    user: 'root',      // Default MySQL user
    password: '2004', // <--- REPLACE WITH YOUR MYSQL WORKBENCH PASSWORD
    database: 'security_system_db', // <--- The new database name
    dateStrings: true,
  timezone: '+08:00'

});

db.connect((err) => {
    if (err) {
        console.error('Error connecting to MySQL:', err);
    } else {
        console.log('Connected to MySQL Database: security_system_db');
    }
});

module.exports = db;
```

```
const express = require('express');
const cors = require('cors');
const bodyParser = require('body-parser');
const db = require('./db');
const nodemailer = require('nodemailer');
const multer = require('multer');
const path = require('path');
const fs = require('fs');
const jwt = require('jsonwebtoken');

// ------------------------------------------------
// SECRETS & IN-MEMORY STORES
// ------------------------------------------------
const JWT_SECRET = 'your-secret-key';
const payrollJWTSecret = 'payroll-secret-key-2026';
const otpStore = {};
const pinAttempts = new Map(); // key: email, value: { count, lastAttempt }

// ------------------------------------------------
```

```
// RATE LIMITER HELPERS (payroll PIN)
// ------------------------------------------------
const MAX_PIN_ATTEMPTS = 5;
const LOCKOUT_DURATION = 15 * 60 * 1000; // 15 minutes

function checkPinRateLimit(email) {
  const now = Date.now();
  if (pinAttempts.has(email)) {
    const { count, lastAttempt } = pinAttempts.get(email);
    if (now - lastAttempt > LOCKOUT_DURATION) {
      pinAttempts.delete(email);
      return true;
    }
    if (count >= MAX_PIN_ATTEMPTS) {
      return false;
    }
  }
  return true;
}

function recordFailedPinAttempt(email) {
  const now = Date.now();
  if (pinAttempts.has(email)) {
    const entry = pinAttempts.get(email);
    entry.count += 1;
    entry.lastAttempt = now;
  } else {
    pinAttempts.set(email, { count: 1, lastAttempt: now });
  }
}

function clearPinAttempts(email) {
  pinAttempts.delete(email);
}

// ------------------------------------------------
// EMAIL TRANSPORTER
// ------------------------------------------------
const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: { user: 'yapkimjean@gmail.com', pass: 'ythz mamn vzal srqv' }
});

// ------------------------------------------------
```

```
// SCHOOL LOCATIONS (still used for some legacy checks)
// ------------------------------------------------
const SCHOOL_LOCATIONS = {
  'HCT Academy Pasig': { lat: 14.57478, lon: 121.06070, radius: 200 },
  'National University - Manila': { lat: 14.6042947, lon: 120.9942832, radius: 200 },
  'Olivarez College Paranaque': { lat: 14.478841, lon: 120.996335, radius: 200 },
  'Wesleyan University Philippines': { lat: 15.484488, lon: 120.976045, radius: 200 },
  'Colegio de San Agustin - Bacolod': { lat: 10.66262, lon: 122.97641, radius: 200 },
  'S Residence Tower 3': { lat: 14.53346, lon: 120.98808, radius: 150 },
  'Sun Residence Tower 1': { lat: 14.61828, lon: 121.00059, radius: 150 }
};

function getDistanceFromLatLonInMeters(lat1, lon1, lat2, lon2) {
  const R = 6371000;
  const dLat = (lat2 - lat1) * Math.PI / 180;
  const dLon = (lon2 - lon1) * Math.PI / 180;
  const a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(lat1 * Math.PI / 180) * Math.cos(lat2 * Math.PI / 180) *
    Math.sin(dLon / 2) * Math.sin(dLon / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return R * c;
}

// ------------------------------------------------
// MULTER SETUP
// ------------------------------------------------
const uploadDir = './uploads/leave_images';
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}
const storage = multer.diskStorage({
  destination: (req, file, cb) => { cb(null, uploadDir); },
  filename: (req, file, cb) => {
    const unique = Date.now() + '-' + Math.round(Math.random() * 1E9);
    const ext = path.extname(file.originalname);
    cb(null, `leave_${unique}${ext}`);
  }
});
const upload = multer({ storage, limits: { fileSize: 5 * 1024 * 1024 } });

const app = express();
app.use(cors());
app.use(bodyParser.json());
app.use('/uploads', express.static('uploads'));
```

```
// ============================================
// 1. AUTHENTICATION & OTP
// ============================================
app.post('/api/auth/send-otp', (req, res) => {
  const email = req.body.email ? req.body.email.trim().toLowerCase() : '';
  if (!email) return res.status(400).json({ success: false, message: 'Email required' });

  db.query("SELECT * FROM users WHERE email = ? AND status = 'active'", [email], (err,
results) => {
    if (err) return res.status(500).json({ success: false, message: err.message });
    if (results.length === 0) return res.status(404).json({ success: false, message: 'No active
account found with that email.' });

    const otp = Math.floor(100000 + Math.random() * 900000).toString();
    const expiresAt = Date.now() + 5 * 60 * 1000;
    otpStore[email] = { otp, expiresAt };
    console.log(`[OTP] ${email} → ${otp}`);

    const mailOptions = {
      from: 'yapkimjean@gmail.com',
      to: email,
      subject: 'Your OTP Code - UniVITA',
      text: `Your OTP code is: ${otp}\n\nThis code expires in 5 minutes.\n\nIf you did not request
this, please ignore this email.`
    };

    transporter.sendMail(mailOptions, (error) => {
      if (error) {
        console.error('OTP email error:', error);
        return res.status(500).json({ success: false, message: 'Failed to send OTP email.' });
      }
      res.json({ success: true, message: 'OTP sent to your email.' });
    });
  });
});

app.post('/api/auth/verify-otp', (req, res) => {
  const email = req.body.email ? req.body.email.trim().toLowerCase() : '';
  const otp = req.body.otp;
  if (!email || !otp) return res.status(400).json({ success: false, message: 'Email and OTP
required' });

  const record = otpStore[email];
```

```
  if (!record) return res.status(400).json({ success: false, message: 'No OTP found. Please request a new one.' });
  if (Date.now() > record.expiresAt) {
    delete otpStore[email];
    return res.status(400).json({ success: false, message: 'OTP has expired. Please request a new one.' });
  }
  if (record.otp !== otp) {
    return res.status(400).json({ success: false, message: 'Invalid OTP.' });
  }

  delete otpStore[email];

  db.query("SELECT * FROM users WHERE email = ? AND status = 'active'", [email], (err, results) => {
    if (err) return res.status(500).json({ success: false, message: err.message });
    if (results.length === 0) return res.status(404).json({ success: false, message: 'User not found.' });

    const user = results[0];
    res.json({
      success: true,
      user: {
        id: user.id,
        employee_id: user.employee_id,
        full_name: user.full_name,
        email: user.email,
        role: user.role,
        monthly_salary: user.monthly_salary || 0,
        work_days_per_month: user.work_days_per_month || 22
      }
    });
  });
});

app.post('/api/login', (req, res) => {
  const { email, password } = req.body;
  const sql = `
    SELECT *, DATEDIFF(CURRENT_DATE, password_last_changed) as days_since_change
    FROM users
    WHERE email = ? AND password = ? AND status = 'active'
  `;
  db.query(sql, [email, password], (err, result) => {
    if (err) return res.status(500).json({ error: err.message });
```

```
    if (result.length > 0) {
      const user = result[0];
      const daysSinceChange = user.days_since_change || 0;
      if (daysSinceChange >= 365) {
        return res.json({
          success: true,
          requiresPasswordReset: true,
          message: "Your password has expired (365+ days old). Please renew it.",
          user: {
            id: user.id,
            employee_id: user.employee_id,
            full_name: user.full_name,
            email: user.email,
            role: user.role
          }
        });
      }
      const tempToken = jwt.sign({ id: user.id, email: user.email }, JWT_SECRET, { expiresIn: '5m'
});
      res.json({
        success: true,
        requiresOtp: true,
        tempToken: tempToken,
        email: user.email
      });
    } else {
      db.query("SELECT status FROM users WHERE email = ?", [email], (err, check) => {
        if (check && check.length > 0 && check[0].status !== 'active') {
          return res.json({ success: false, message: "Your account has been deactivated." });
        }
        res.json({ success: false, message: "Invalid email or password" });
      });
    }
  });
});

app.put('/api/users/:identifier/update-password', (req, res) => {
  const { identifier } = req.params;
  const { currentPassword, newPassword } = req.body;
  const sql = "SELECT * FROM users WHERE (id = ? OR employee_id = ?)";
  db.query(sql, [identifier, identifier], (err, results) => {
    if (err) return res.status(500).json({ success: false, message: "DB Error" });
    if (results.length === 0) return res.status(404).json({ success: false, message: "User not
found." });
```

```
    const user = results[0];
    if (user.password.toString().trim() !== currentPassword.toString().trim()) {
      return res.status(401).json({ success: false, message: "Invalid current password." });
    }
    const updateSql = "UPDATE users SET password = ?, password_last_changed =
CURRENT_DATE WHERE id = ?";
    db.query(updateSql, [newPassword.trim(), user.id], (err) => {
      if (err) return res.status(500).json({ success: false });
      res.json({ success: true });
    });
  });
});

// ============================================
// 2. ATTENDANCE & CLOCKING
// ============================================
const getPHTime = () => {
  const now = new Date();
  const date = now.toLocaleDateString('en-CA');
  const time = now.toLocaleTimeString('en-GB', { hour12: false });
  return { date, time };
};

app.post('/api/clock-in', async (req, res) => {
  const { user_id, location } = req.body;
  const { date: todayDate, time: currentTime } = getPHTime();
  try {
    const [schedRows] = await db.promise().query(
      "SELECT place, start_time FROM schedules WHERE user_id = ? AND date = ?",
      [user_id, todayDate]
    );
    if (schedRows.length === 0) {
      return res.status(403).json({ success: false, message: "No work schedule assigned for
today." });
    }
    const { place: schedulePlace, start_time: scheduledStartTime } = schedRows[0];

    const [rows] = await db.promise().query(
      "SELECT latitude AS lat, longitude AS lon, radius FROM school_locations WHERE name =
?",
      [schedulePlace]
    );
    if (rows.length === 0) {
```

```
      return res.status(400).json({ success: false, message: `Schedule location
"${schedulePlace}" is not recognized.` });
    }
    const schoolGeo = rows[0];

    const { lat, lon } = location;
    if (!lat || !lon) {
      return res.status(400).json({ success: false, message: "GPS coordinates missing." });
    }

    const distance = getDistanceFromLatLonInMeters(lat, lon, schoolGeo.lat, schoolGeo.lon);
    if (distance > schoolGeo.radius) {
      return res.status(403).json({ success: false, message: `Not within ${schedulePlace} campus.
Distance: ${Math.round(distance)}m` });
    }

    if (currentTime < scheduledStartTime) {
      return res.status(403).json({ success: false, message: `Clock-in allowed only from
${scheduledStartTime}.` });
    }

    const [existing] = await db.promise().query(
      "SELECT * FROM attendance WHERE user_id = ? AND date = ?",
      [user_id, todayDate]
    );
    if (existing.length > 0) {
      return res.json({ success: false, message: "Already clocked in today." });
    }

    const finalStatus = currentTime > scheduledStartTime ? 'Late' : 'Present';
    const locationString = location.address || `${lat},${lon}`;
    await db.promise().query(
      "INSERT INTO attendance (user_id, date, time_in, status, location) VALUES (?, ?, ?, ?, ?)",
      [user_id, todayDate, currentTime, finalStatus, locationString]
    );
    res.json({ success: true, message: `Clock-in successful as ${finalStatus} at ${currentTime}!`
});
  } catch (err) {
    console.error("Clock-in error:", err);
    res.status(500).json({ success: false, error: err.message });
  }
});

app.post('/api/clock-out', async (req, res) => {
```

```
  const { user_id, location } = req.body;
  const { date: todayDate, time: currentTime } = getPHTime();
  try {
    const [schedRows] = await db.promise().query(
      "SELECT place, end_time FROM schedules WHERE user_id = ? AND date = ?",
      [user_id, todayDate]
    );
    if (schedRows.length === 0) {
      return res.status(403).json({ success: false, message: "No work schedule found for today."
});
    }
    const { place: schedulePlace, end_time: scheduledEndTime } = schedRows[0];

    const [rows] = await db.promise().query(
      "SELECT latitude AS lat, longitude AS lon, radius FROM school_locations WHERE name =
?",
      [schedulePlace]
    );
    if (rows.length === 0) {
      return res.status(400).json({ success: false, message: `Schedule location
"${schedulePlace}" is not recognized.` });
    }
    const schoolGeo = rows[0];

    const { lat, lon } = location;
    if (!lat || !lon) {
      return res.status(400).json({ success: false, message: "GPS coordinates missing." });
    }

    const distance = getDistanceFromLatLonInMeters(lat, lon, schoolGeo.lat, schoolGeo.lon);
    if (distance > schoolGeo.radius) {
      return res.status(403).json({ success: false, message: `Not within ${schedulePlace} campus.
Distance: ${Math.round(distance)}m` });
    }

    if (currentTime < scheduledEndTime) {
      return res.status(403).json({ success: false, message: `Shift incomplete. You must stay until
${scheduledEndTime} to clock out.` });
    }

    const locationString = location.address || `${lat},${lon}`;
    const [result] = await db.promise().query(
      "UPDATE attendance SET time_out = ?, location = ? WHERE user_id = ? AND date = ?
AND time_out IS NULL",
```

```
      [currentTime, locationString, user_id, todayDate]
    );
    if (result.affectedRows === 0) {
      return res.json({ success: false, message: "No active clock-in found or already clocked out."
});
    }
    res.json({ success: true, message: `Clock-out successful at ${currentTime}!` });
  } catch (err) {
    console.error("Clock-out error:", err);
    res.status(500).json({ success: false, error: err.message });
  }
});

// ============================================
// 3. ATTENDANCE APPEALS
// ============================================
const appealUploadDir = './uploads/attendance_appeals';
if (!fs.existsSync(appealUploadDir)) fs.mkdirSync(appealUploadDir, { recursive: true });
const appealStorage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, appealUploadDir),
  filename: (req, file, cb) => {
    const unique = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, `appeal_${unique}${path.extname(file.originalname)}`);
  }
});
const uploadAppeal = multer({ storage: appealStorage, limits: { fileSize: 5 * 1024 * 1024 } });

// 1. Mobile submission – employee requests appeal (pending)
app.post('/api/attendance-appeals', uploadAppeal.single('image'), (req, res) => {
  const { user_id, date, reason } = req.body;
  let image_url = null;
  if (req.file) {
    image_url = `/uploads/attendance_appeals/${req.file.filename}`;
  }
  const sql = "INSERT INTO attendance_appeals (user_id, date, reason, image_url, status)
VALUES (?, ?, ?, ?, 'pending')";
  db.query(sql, [user_id, date, reason, image_url], (err) => {
    if (err) return res.status(500).json({ success: false, error: err.message });
    res.json({ success: true });
  });
});

// 2. Admin manual appeal – immediate insertion (approved)
app.post('/api/attendance-appeal', uploadAppeal.single('image'), (req, res) => {
```

```
  const { user_id, date, time_in, time_out, reason } = req.body;
  let image_url = null;
  if (req.file) image_url = `/uploads/attendance_appeals/${req.file.filename}`;

  // First, fetch schedule to get proper times and total hours
  const getScheduleSql = "SELECT start_time, end_time FROM schedules WHERE user_id = ?
AND date = ?";
  db.query(getScheduleSql, [user_id, date], (err, scheduleRows) => {
    let start_time = time_in;
    let end_time = time_out;
    let total_hours = 0;
    if (scheduleRows.length > 0) {
      const schedStart = scheduleRows[0].start_time;
      const schedEnd = scheduleRows[0].end_time;
      // If admin didn't provide times, use schedule times
      if (!time_in && schedStart) start_time = schedStart;
      if (!time_out && schedEnd) end_time = schedEnd;
      if (start_time && end_time) {
        const start = new Date(`1970-01-01T${start_time}`);
        const end = new Date(`1970-01-01T${end_time}`);
        total_hours = (end - start) / (1000 * 3600);
      }
    } else {
      // No schedule – just use provided times (if any) and compute hours
      if (time_in && time_out) {
        const start = new Date(`1970-01-01T${time_in}`);
        const end = new Date(`1970-01-01T${time_out}`);
        total_hours = (end - start) / (1000 * 3600);
      }
    }

    // Insert or update attendance record
    const upsertSql = `
      INSERT INTO attendance (user_id, date, time_in, time_out, status, location, total_hours)
      VALUES (?, ?, ?, ?, 'Present', 'Admin Appeal', ?)
      ON DUPLICATE KEY UPDATE
        time_in = VALUES(time_in),
        time_out = VALUES(time_out),
        status = 'Present',
        location = 'Admin Appeal',
        total_hours = VALUES(total_hours)
    `;
    db.query(upsertSql, [user_id, date, start_time, end_time, total_hours], (err) => {
      if (err) return res.status(500).json({ success: false, error: err.message });
```

```
      // Log the appeal as approved (override if duplicate)
      const appealSql = `
        INSERT INTO attendance_appeals (user_id, date, reason, image_url, status)
        VALUES (?, ?, ?, ?, 'approved')
        ON DUPLICATE KEY UPDATE reason = VALUES(reason), image_url =
VALUES(image_url), status = 'approved'
      `;
      db.query(appealSql, [user_id, date, reason, image_url], (err) => {
        if (err) console.error("Appeal log error:", err);
        res.json({ success: true });
      });
    });
  });
});
// 3. Get pending appeals (for admin modal)
app.get('/api/attendance-appeals/pending', (req, res) => {
  const sql = `
    SELECT a.*, u.full_name, u.employee_id
    FROM attendance_appeals a
    JOIN users u ON a.user_id = u.employee_id
    WHERE a.status = 'pending'
    ORDER BY a.submitted_at DESC
  `;
  db.query(sql, (err, results) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(results);
  });
});

// 4. Update appeal status (approve/reject) – automatically fill attendance with schedule times
app.put('/api/attendance-appeals/:id/status', (req, res) => {
  const { status, admin_remarks } = req.body;
  const appealId = req.params.id;

  // 1. Get appeal details
  const getAppealSql = "SELECT user_id, date FROM attendance_appeals WHERE id = ?";
  db.query(getAppealSql, [appealId], (err, rows) => {
    if (err) return res.status(500).json({ error: err.message });
    if (rows.length === 0) return res.status(404).json({ error: "Appeal not found" });
    const { user_id, date } = rows[0];

    // 2. Get schedule for that day (to know shift times)
```

```
    const getScheduleSql = "SELECT start_time, end_time FROM schedules WHERE user_id =
? AND date = ?";
    db.query(getScheduleSql, [user_id, date], (err, scheduleRows) => {
      let start_time = null, end_time = null, total_hours = 0;
      if (scheduleRows.length > 0) {
        start_time = scheduleRows[0].start_time;
        end_time = scheduleRows[0].end_time;
        if (start_time && end_time) {
          const start = new Date(`1970-01-01T${start_time}`);
          const end = new Date(`1970-01-01T${end_time}`);
          total_hours = (end - start) / (1000 * 3600);
        }
      }

      // 3. Update appeal status
      const updateAppealSql = "UPDATE attendance_appeals SET status = ?, admin_remarks =
? WHERE id = ?";
      db.query(updateAppealSql, [status, admin_remarks || null, appealId], (err) => {
        if (err) return res.status(500).json({ error: err.message });

        if (status === 'approved') {
          // 4. Insert or update attendance record using schedule times
          const upsertSql = `
            INSERT INTO attendance (user_id, date, time_in, time_out, status, location, total_hours)
            VALUES (?, ?, ?, ?, 'Present', 'Appeal Approved', ?)
            ON DUPLICATE KEY UPDATE
              time_in = VALUES(time_in),
              time_out = VALUES(time_out),
              status = 'Present',
              location = 'Appeal Approved',
              total_hours = VALUES(total_hours)
          `;
          db.query(upsertSql, [user_id, date, start_time, end_time, total_hours], (err) => {
            if (err) console.error("Failed to upsert attendance:", err);
            res.json({ success: true });
          });
        } else {
          res.json({ success: true });
        }
      });
    });
  });
});
```

```
// Get appeal history (approved and rejected)
app.get('/api/attendance-appeals/history', (req, res) => {
  const sql = `
    SELECT a.*, u.full_name, u.employee_id
    FROM attendance_appeals a
    JOIN users u ON a.user_id = u.employee_id
    WHERE a.status IN ('approved', 'rejected')
    ORDER BY a.submitted_at DESC
  `;
  db.query(sql, (err, results) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(results);
  });
});



// ============================================
// 3. LEAVE REQUESTS
// ============================================
app.post('/api/leave-requests', upload.single('image'), (req, res) => {
    const { user_id, request_date, reason, type } = req.body;
    let image_url = null;
    if (req.file) {
        image_url = `/uploads/leave_images/${req.file.filename}`;
    }
    const sql = "INSERT INTO leave_requests (user_id, request_date, reason, type, status,
is_hidden, image_url) VALUES (?, ?, ?, ?, 'Pending', 0, ?)";
    db.query(sql, [user_id, request_date, reason, type, image_url], (err, result) => {
        if (err) return res.status(500).json({ success: false, error: err.message });
        res.json({ success: true, image_url });
    });
});

app.get('/api/leave-requests/all', (req, res) => {
    const today = new Date().toISOString().slice(0, 10);
    const updateSql = "UPDATE leave_requests SET status = 'Rejected' WHERE status =
'Pending' AND request_date < ?";
    db.query(updateSql, [today], (err) => {
        if (err) console.error("Auto-reject error:", err);
        const selectSql = `
            SELECT lr.*, u.full_name
            FROM leave_requests lr
            LEFT JOIN users u ON lr.user_id = u.employee_id
```

```
            WHERE lr.is_hidden = 0
            ORDER BY lr.request_date DESC
        `;
        db.query(selectSql, (err, result) => {
            if (err) return res.status(500).json(err);
            res.json(result || []);
        });
    });
});

app.get('/api/leave-requests/history/:identifier', (req, res) => {
    const { identifier } = req.params;
    const sql = `
        SELECT lr.*, DATE_FORMAT(lr.request_date, '%Y-%m-%d') as formatted_date
        FROM leave_requests lr
        JOIN users u ON (lr.user_id = u.employee_id OR lr.user_id = u.id)
        WHERE (u.employee_id = ? OR u.id = ?)
        AND lr.status IN ('Approved', 'Rejected')
        AND lr.is_hidden = 0
        ORDER BY lr.request_date DESC
    `;
    db.query(sql, [identifier, identifier], (err, results) => {
        if (err) return res.status(500).json({ success: false, message: err.message });
        res.json(results);
    });
});

app.get('/api/leave-requests/user/:employeeId', (req, res) => {
    const sql = "SELECT * FROM leave_requests WHERE user_id = ? ORDER BY request_date
DESC";
    db.query(sql, [req.params.employeeId], (err, result) => {
        if (err) return res.status(500).json(err);
        res.json(result || []);
    });
});

app.put('/api/leave-requests/:id/status', (req, res) => {
    const { status } = req.body;
    const requestId = req.params.id;
    db.query("SELECT user_id, request_date FROM leave_requests WHERE id = ?", [requestId],
(err, results) => {
        if (err || results.length === 0) return res.status(500).json({ error: "Request not found" });
        const { user_id, request_date } = results[0];
```

```
        db.query("UPDATE leave_requests SET status = ? WHERE id = ?", [status, requestId],
(err) => {
            if (err) return res.status(500).json(err);
            if (status === 'Approved') {
                const attendanceSql = `
                    INSERT INTO attendance (user_id, date, status, location)
                    VALUES (?, ?, 'on leave', 'Remote/Leave')
                    ON DUPLICATE KEY UPDATE status = 'on leave'`;
                db.query(attendanceSql, [user_id, request_date]);
            }
            res.json({ success: true });
        });
    });
});

app.put('/api/leave-requests/:id/dismiss', (req, res) => {
    const sql = "UPDATE leave_requests SET is_hidden = 1 WHERE id = ?";
    db.query(sql, [req.params.id], (err, result) => {
        if (err) return res.status(500).json({ success: false, error: err.message });
        res.json({ success: true, message: "Request hidden" });
    });
});

// ============================================
// 4. CALENDAR & SHARED EVENTS
// ============================================
app.get('/api/events', (req, res) => {
    const sql = `
        SELECT id, title, DATE_FORMAT(date, '%Y-%m-%d') as date, place, start_time, end_time,
type, description, 'None' as status
        FROM events
        UNION ALL
        SELECT
            lr.id,
            CONCAT(u.full_name, ' (', lr.type, ')') as title,
            DATE_FORMAT(lr.request_date, '%Y-%m-%d') as date,
            'Leave' as place,
            '08:00:00' as start_time,
            '17:00:00' as end_time,
            lr.type as type,
            lr.reason as description,
            lr.status
        FROM leave_requests lr
        JOIN users u ON (lr.user_id = u.employee_id OR lr.user_id = u.id)
```

```
        WHERE lr.is_hidden = 0
        ORDER BY date ASC
    `;
    db.query(sql, (err, results) => {
        if (err) return res.status(500).send(err);
        res.json(results);
    });
});

app.post('/api/events', (req, res) => {
    const { title, date, place, start_time, end_time, type, description } = req.body;
    const sql = `INSERT INTO events (title, date, place, start_time, end_time, type, description)
VALUES (?, ?, ?, ?, ?, ?, ?)`;
    db.query(sql, [title, date, place, start_time, end_time, type, description], (err) => {
        if (err) return res.status(500).json({ success: false, error: err.message });
        res.json({ success: true });
    });
});

app.put('/api/events/:id', (req, res) => {
    const { title, date, place, start_time, end_time, type, description } = req.body;
    const sql = `UPDATE events SET title=?, date=?, place=?, start_time=?, end_time=?, type=?,
description=? WHERE id=?`;
    db.query(sql, [title, date, place, start_time, end_time, type, description, req.params.id], (err)
=> {
        if (err) return res.status(500).json({ success: false, error: err.message });
        res.json({ success: true });
    });
});

app.delete('/api/events/:id', (req, res) => {
    db.query("DELETE FROM events WHERE id = ?", [req.params.id], (err) => {
        if (err) return res.status(500).json({ success: false, error: err.message });
        res.json({ success: true });
    });
});

// ============================================
// 5. SCHEDULING
// ============================================
app.get('/api/schedules', (req, res) => {
    const sql = `
```

```
      SELECT u.full_name, u.employee_id, s.id AS schedule_id, DATE_FORMAT(s.date,
'%Y-%m-%d') AS schedule_date, s.place, s.course, s.start_time, s.end_time,
COALESCE(a.status, 'No Record') AS attendance_status
      FROM users u
      INNER JOIN schedules s ON u.employee_id = s.user_id
      LEFT JOIN attendance a ON u.employee_id = a.user_id AND DATE(a.date) =
DATE(s.date)
      WHERE LOWER(u.role) = 'instructor'
      UNION
      SELECT u.full_name, u.employee_id, NULL AS schedule_id, DATE_FORMAT(a.date,
'%Y-%m-%d') AS schedule_date, 'Remote/Leave' AS place, 'On Leave' AS course, '00:00:00'
AS start_time, '00:00:00' AS end_time, a.status AS attendance_status
      FROM users u
      JOIN attendance a ON u.employee_id = a.user_id
      WHERE a.status = 'on leave' AND LOWER(u.role) = 'instructor'
      ORDER BY schedule_date DESC, full_name ASC
    `;
    db.query(sql, (err, result) => {
      if (err) return res.status(500).json({ error: err.message });
      res.json(result);
    });
});

app.get('/api/schedules/:employeeId', (req, res) => {
    const sql = `SELECT *, DATE_FORMAT(date, '%Y-%m-%d') as date FROM schedules
WHERE user_id = ? ORDER BY date ASC, start_time ASC`;
    db.query(sql, [req.params.employeeId], (err, result) => {
      if (err) return res.status(500).json({ error: err.message });
      res.json(result || []);
    });
});

// Helper function to check overlapping schedules
async function hasScheduleConflict(user_id, date, start_time, end_time, excludeId = null) {
  let sql = `SELECT id FROM schedules
            WHERE user_id = ? AND date = ?
            AND NOT (end_time <= ? OR start_time >= ?)`;
  const params = [user_id, date, start_time, end_time];
  const [rows] = await db.promise().query(sql, params);
  if (excludeId) {
    return rows.filter(r => r.id != excludeId).length > 0;
  }
  return rows.length > 0;
}
```

```
// POST schedule
app.post('/api/schedules', async (req, res) => {
  const { user_id, date, place, course, start_time, end_time } = req.body;
  const today = new Date().toISOString().split('T')[0];
  if (date < today) {
    return res.status(400).json({ success: false, error: "Cannot create schedule for a past date."
});
  }


  const [locRows] = await db.promise().query("SELECT id FROM school_locations WHERE
name = ?", [place]);
  if (locRows.length === 0) {
    return res.status(400).json({ success: false, error: "Schedule place must be a registered
school location." });
  }

  if (start_time >= end_time) {
    return res.status(400).json({ success: false, error: "End time must be after start time." });
  }
  if (await hasScheduleConflict(user_id, date, start_time, end_time)) {
    return res.status(409).json({ success: false, error: "Time conflict: Instructor already has a
schedule overlapping with these times." });
  }
  const sql = "INSERT INTO schedules (user_id, date, place, course, start_time, end_time)
VALUES (?, ?, ?, ?, ?, ?)";
  db.query(sql, [user_id, date, place, course, start_time, end_time], (err) => {
    if (err) return res.status(500).json({ success: false });
    res.json({ success: true });
  });
});

// PUT schedule
app.put('/api/schedules/:id', async (req, res) => {
  const { date, place, course, start_time, end_time } = req.body;
  const scheduleId = req.params.id;
  const today = new Date().toISOString().split('T')[0];
  if (date < today) {
    return res.status(400).json({ success: false, error: "Cannot update schedule to a past date."
});
  }

  if (place) {
```

```
    const [locRows] = await db.promise().query("SELECT id FROM school_locations WHERE
name = ?", [place]);
    if (locRows.length === 0) {
      return res.status(400).json({ success: false, error: "Schedule place must be a registered
school location." });
    }
  }

  if (start_time >= end_time) {
    return res.status(400).json({ success: false, error: "End time must be after start time." });
  }
  const [old] = await db.promise().query("SELECT user_id FROM schedules WHERE id = ?",
[scheduleId]);
  if (old.length === 0) return res.status(404).json({ success: false });
  const user_id = old[0].user_id;
  if (await hasScheduleConflict(user_id, date, start_time, end_time, scheduleId)) {
    return res.status(409).json({ success: false, error: "Time conflict: Instructor already has a
schedule overlapping with these times." });
  }
  const sql = "UPDATE schedules SET date=?, place=?, course=?, start_time=?, end_time=?
WHERE id=?";
  db.query(sql, [date, place, course, start_time, end_time, scheduleId], (err) => {
    if (err) return res.status(500).json({ success: false });
    res.json({ success: true });
  });
});

app.delete('/api/schedules/:id', (req, res) => {
    db.query("DELETE FROM schedules WHERE id = ?", [req.params.id], (err) => {
        if (err) return res.status(500).json({ success: false });
        res.json({ success: true });
    });
});

// ==========================================
// 6. REPORTS & EMPLOYEES
// ==========================================

// Admin resets another user's password (does not require old password)
app.put('/api/users/:id/reset-password', (req, res) => {
  const { newPassword } = req.body;
  const userId = req.params.id;

  if (!newPassword || newPassword.trim().length < 6) {
```

```
    return res.status(400).json({ success: false, message: 'Password must be at least 6
characters.' });
  }

  const sql = 'UPDATE users SET password = ?, password_last_changed = CURRENT_DATE
WHERE id = ?';
  db.query(sql, [newPassword.trim(), userId], (err, result) => {
    if (err) return res.status(500).json({ success: false, message: err.message });
    if (result.affectedRows === 0) return res.status(404).json({ success: false, message: 'User not
found.' });
    res.json({ success: true, message: 'Password reset successfully.' });
  });
});
app.get('/api/employees', (req, res) => {
  db.query("SELECT * FROM users ORDER BY full_name ASC", (err, result) => {
    if (err) return res.status(500).json(err);
    res.json(result || []);
  });
});

// --- Create employee (now includes payroll_access and payroll_pin) ---
app.post('/api/employees', (req, res) => {
  const {
    employee_id, full_name, email, password, role,
    employment_type, position_level, contract_type,
    monthly_salary, work_days_per_month,
    payroll_access, payroll_pin
  } = req.body;

  const finalPin = payroll_pin || '1234';
  const finalAccess = payroll_access || 0;

  const sql = `INSERT INTO users
    (employee_id, full_name, email, password, role, status,
     employment_type, position_level, contract_type,
     monthly_salary, work_days_per_month,
     payroll_access, payroll_pin)
    VALUES (?, ?, ?, ?, ?, 'active', ?, ?, ?, ?, ?, ?, ?)`;

  db.query(sql, [
    employee_id, full_name, email, password, role,
    employment_type, position_level, contract_type,
    monthly_salary, work_days_per_month,
    finalAccess, finalPin
```

```
  ], (err) => {
    if (err) return res.status(500).json({ success: false, error: err.message });
    res.json({ success: true });
  });
});

// --- Update employee (supports payroll_access and payroll_pin) ---
app.put('/api/employees/:id', (req, res) => {
  const {
    full_name, role, employment_type, position_level,
    contract_type, monthly_salary, work_days_per_month,
    payroll_access, payroll_pin
  } = req.body;

  const fields = [
    full_name, role, employment_type, position_level,
    contract_type, monthly_salary, work_days_per_month
  ];
  let sql = `UPDATE users SET
    full_name=?, role=?, employment_type=?, position_level=?,
    contract_type=?, monthly_salary=?, work_days_per_month=?`;

  if (payroll_access !== undefined) {
    sql += `, payroll_access=?`;
    fields.push(payroll_access ? 1 : 0);
  }
  if (payroll_pin !== undefined) {
    sql += `, payroll_pin=?`;
    fields.push(payroll_pin);
  }

  sql += ` WHERE id=?`;
  fields.push(req.params.id);

  db.query(sql, fields, (err) => {
    if (err) return res.status(500).json({ success: false, error: err.message });
    res.json({ success: true });
  });
});

app.delete('/api/employees/:id', (req, res) => {
    db.query("DELETE FROM users WHERE id = ? AND LOWER(role) = 'instructor'",
[req.params.id], (err, result) => {
        if (err) return res.status(500).json({ success: false, error: err.message });
```

```
        if (result.affectedRows === 0) return res.status(404).json({ success: false, message:
"Employee not found" });
        res.json({ success: true });
    });
});

app.put('/api/employees/:employeeId/toggle-status', (req, res) => {
    const { employeeId } = req.params;
    db.query("SELECT status FROM users WHERE employee_id = ?", [employeeId], (err,
results) => {
        if (err || results.length === 0) return res.status(500).json({ success: false });
        const newStatus = results[0].status === 'active' ? 'deactivated' : 'active';
        db.query("UPDATE users SET status = ? WHERE employee_id = ?", [newStatus,
employeeId], (err) => {
            if (err) return res.status(500).json({ success: false });
            res.json({ success: true, newStatus });
        });
    });
});

app.get('/api/attendance-report', (req, res) => {
    const { date } = req.query;
    // NOTE: salary_rate and overtime_rate have been dropped from the users table.
    // All financial calculations now use monthly_salary and work_days_per_month.
    const sql = `
        SELECT
            u.id,
            u.full_name,
            u.employee_id,
            a.status,
            a.time_in,
            a.time_out,
            a.location,
            s.start_time AS scheduled_start,
            s.end_time AS scheduled_end,
            DATE_FORMAT(COALESCE(a.date, s.date, ?), '%Y-%m-%d') AS attendance_date,
            COALESCE(ROUND(TIMESTAMPDIFF(MINUTE, a.time_in, a.time_out) / 60, 2), 0) AS
total_hours,
            COALESCE(ROUND((TIMESTAMPDIFF(MINUTE, a.time_in, a.time_out) / 60) *
(u.monthly_salary / u.work_days_per_month / 8), 2), 0) AS gross_pay,
            COALESCE(ROUND(((TIMESTAMPDIFF(MINUTE, a.time_in, a.time_out) / 60) *
(u.monthly_salary / u.work_days_per_month / 8)) * 0.10, 2), 0) AS tax_deduction,
            COALESCE(ROUND(((TIMESTAMPDIFF(MINUTE, a.time_in, a.time_out) / 60) *
(u.monthly_salary / u.work_days_per_month / 8)) * 0.90, 2), 0) AS net_pay
```

```
      FROM users u
      LEFT JOIN schedules s ON u.employee_id = s.user_id AND DATE(s.date) = DATE(?)
      LEFT JOIN attendance a ON u.employee_id = a.user_id AND DATE(a.date) = DATE(?)
      WHERE LOWER(u.role) = 'instructor'
      ORDER BY u.full_name ASC
    `;
    db.query(sql, [date, date, date], (err, result) => {
      if (err) return res.status(500).json({ error: err.message });
      res.json(result || []);
    });
});

app.get('/api/attendance-report-user/:employeeId', (req, res) => {
    const sql = "SELECT *, DATE_FORMAT(date, '%Y-%m-%d') as date,
ROUND(TIMESTAMPDIFF(MINUTE, time_in, time_out) / 60, 2) as total_hours FROM
attendance WHERE user_id = ? ORDER BY date DESC";
    db.query(sql, [req.params.employeeId], (err, result) => {
      if (err) return res.status(500).json({ error: err.message });
      res.json(result || []);
    });
});

// ==========================================
// 7. APPOINTMENTS & VISITOR MANAGEMENT
// ==========================================

// Get additional visitors for a specific appointment
app.get('/api/appointments/:id/visitors', (req, res) => {
  const { id } = req.params;
  db.query("SELECT * FROM appointment_visitors WHERE appointment_id = ?", [id], (err,
results) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(results);
  });
});



app.get('/api/ble-tags', (req, res) => {
  db.query("SELECT * FROM ble_tags ORDER BY label", (err, results) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(results);
  });
});
```

```
app.post('/api/ble-tags', (req, res) => {
  const { ble_id, label, mac_address } = req.body;
  if (!ble_id) return res.status(400).json({ error: "BLE ID is required." });
  if (!mac_address) return res.status(400).json({ error: "MAC address is required." });   // ← new
check

  const sql = "INSERT INTO ble_tags (ble_id, label, mac_address) VALUES (?, ?, ?)";
  db.query(sql, [ble_id, label || '', mac_address], (err, result) => {
    if (err) {
      if (err.code === 'ER_DUP_ENTRY') return res.status(400).json({ error: "Tag already exists."
});
      return res.status(500).json({ error: err.message });
    }
    res.json({ success: true, id: result.insertId });
  });
});
// Delete a BLE tag
app.delete('/api/ble-tags/:id', (req, res) => {
  const { id } = req.params;
  db.query("DELETE FROM ble_tags WHERE id = ?", [id], (err, result) => {
    if (err) return res.status(500).json({ error: err.message });
    if (result.affectedRows === 0) return res.status(404).json({ error: "Tag not found." });
    res.json({ success: true });
  });
});


app.get('/api/ble-tags/in-use', (req, res) => {
  const today = new Date().toISOString().split('T')[0];


  const sql = `
    SELECT DISTINCT ble_id FROM visitor_requests
    WHERE status = 'APPROVED' AND visit_date >= ?
    UNION
    SELECT DISTINCT ble_id FROM appointment_visitors av
    JOIN visitor_requests vr ON av.appointment_id = vr.id
    WHERE vr.status = 'APPROVED' AND vr.visit_date >= ?
  `;
  db.query(sql, [today, today], (err, results) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(results.map(r => r.ble_id));
  });
```

```
});

// ----- UPDATED: Book appointment with multi-visitor support and primary BLE tag -----
app.post('/api/appointments/book', (req, res) => {
  const {
    firstName, lastName, email, phone, date, time, reason,
    primaryBleId,           // <-- new field from frontend
    additionalVisitors      // array of { name, bleId }
  } = req.body;

  const today = new Date().toISOString().split('T')[0];
  if (date < today) {
    return res.status(400).json({ success: false, error: "Cannot book appointments for past dates."
});
  }

  const sql = `INSERT INTO visitor_requests
    (first_name, last_name, email, phone, visit_date, visit_time, reason, ble_id, status)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'PENDING')`;
  db.query(sql, [firstName, lastName, email, phone, date, time, reason, primaryBleId || null], (err,
result) => {
    if (err) return res.status(500).json({ success: false, error: err.message });
    const appointmentId = result.insertId;

    // Insert additional visitors (if any)
    if (additionalVisitors && additionalVisitors.length > 0) {
      const visSql = "INSERT INTO appointment_visitors (appointment_id, visitor_name, ble_id)
VALUES ?";
      const values = additionalVisitors.map(v => [appointmentId, v.name, v.bleId || null]);
      db.query(visSql, [values], (err2) => {
        if (err2) console.error("Error inserting additional visitors:", err2);
      });
    }

    // Build confirmation email with BLE tag details
    const primaryTagText = primaryBleId ? `Your BLE Tag: ${primaryBleId}\n` : '';
    const phoneText = phone ? `Phone: ${phone}\n` : '';
    let visitorDetails = '';
    if (additionalVisitors && additionalVisitors.length > 0) {
      visitorDetails = '\nAdditional Visitors:\n';
      additionalVisitors.forEach((v, i) => {
        visitorDetails += `  ${i + 1}. ${v.name} – BLE Tag: ${v.bleId || 'None assigned'}\n`;
      });
```

```
      visitorDetails += '\nPlease share the BLE tags with your companions. They will be given to you upon arrival by our security guard.\n';
    }

    const mailOptions = {
      from: 'yapkimjean@gmail.com',
      to: email,
      subject: 'Visit Request Received - HCT Academy',
      text: `Dear ${firstName} ${lastName},\n\nYour visit request has been received. We will review it and notify you once approved.\n\nDetails:\nDate: ${date}\nTime: ${time}\n${phoneText}${primaryTagText}Reason: ${reason}\n${visitorDetails}\nThank you,\nHCT Academy`
    };

    transporter.sendMail(mailOptions, (error) => {
      if (error) {
        console.error("Email error:", error);
        return res.json({ success: true, emailSent: false, message: "Request saved but email failed." });
      }
      res.json({ success: true, emailSent: true });
    });
  });
});

// ----- UPDATED: Approve / Reject appointment (primary BLE tag in email) -----
app.put('/api/appointments/:id/status', (req, res) => {
  const { status, adminNotes, adminId } = req.body;
  const requestId = req.params.id;

  const checkSql = "SELECT * FROM visitor_requests WHERE id = ?";
  db.query(checkSql, [requestId], (err, results) => {
    if (err) return res.status(500).json({ error: err.message });
    if (results.length === 0) return res.status(404).json({ error: "Request not found" });

    const request = results[0];
    const { first_name, last_name, email, visit_date, visit_time, reason, phone, ble_id } = request;

    const updateSql = `
      UPDATE visitor_requests
      SET status = ?, admin_notes = ?, processed_by = ?, processed_at = NOW()
      WHERE id = ?
    `;
    db.query(updateSql, [status, adminNotes || null, adminId, requestId], (err) => {
```

```
    if (err) return res.status(500).json({ error: err.message });

    // Fetch additional visitors
    db.query("SELECT * FROM appointment_visitors WHERE appointment_id = ?", [requestId],
(err, visitors) => {
      if (err) visitors = [];

      const isApproved = status === 'APPROVED';
      const subject = isApproved
        ? "Visit Request Approved - HCT Academy"
        : "Visit Request Status Update - HCT Academy";

      let emailBody = `Dear ${first_name} ${last_name},\n\n`;
      if (isApproved) {
        emailBody += "Your visit request has been APPROVED.\n\nWe look forward to
welcoming you to HCT Academy.\n\n";
      } else {
        emailBody += "We regret to inform you that your visit request has been DECLINED.\n\n";
        if (adminNotes) emailBody += `Reason: ${adminNotes}\n\n`;
      }

      emailBody += `Details:\nDate: ${visit_date}\nTime: ${visit_time}\nReason: ${reason}\n`;
      if (phone) emailBody += `Phone: ${phone}\n`;
      if (ble_id) emailBody += `Primary BLE Tag: ${ble_id}\n`;

      if (visitors && visitors.length > 0) {
        emailBody += "\nAdditional Visitors / BLE Tags:\n";
        visitors.forEach((v, i) => {
          emailBody += `  ${i + 1}. ${v.visitor_name} – BLE Tag: ${v.ble_id || 'None assigned'}\n`;
        });
        emailBody += "\nPlease remind your companions to bring their assigned BLE tags. They
will be used for tracking inside the building.\n";
      }

      emailBody += "\nThank you for your understanding.\n\nBest regards,\nHCT Academy";

      const mailOptions = {
        from: 'yapkimjean@gmail.com',
        to: email,
        subject: subject,
        text: emailBody
      };

      transporter.sendMail(mailOptions, (error) => {
```

```
        if (error) {
          console.error("Email error:", error);
          return res.json({ success: true, emailSent: false, message: "Status updated but email
failed." });
        }
        res.json({ success: true, emailSent: true });
      });
    });
  });
});

app.get('/api/appointments/pending', (req, res) => {
  db.query("SELECT * FROM visitor_requests WHERE status = 'PENDING'", (err, results) => {
    if (err) return res.status(500).json(err);
    res.send(results);
  });
});

app.get('/api/appointments/history', (req, res) => {
  db.query("SELECT * FROM visitor_requests WHERE status != 'PENDING' ORDER BY
visit_date DESC", (err, results) => {
    if (err) return res.status(500).json(err);
    res.json(results);
  });
});

app.get('/api/dashboard/summary', async (req, res) => {
  try {
    const [leaves, visitors, employees] = await Promise.all([
      db.promise().query("SELECT COUNT(*) as count FROM leave_requests WHERE status
= 'Pending'"),
      db.promise().query("SELECT COUNT(*) as count FROM attendance WHERE date =
CURDATE()"),
      db.promise().query("SELECT COUNT(*) as count FROM events WHERE date =
CURDATE()")
    ]);
    res.json({
      pendingLeaves: leaves[0][0].count,
      presentToday: visitors[0][0].count,
      eventsToday: employees[0][0].count
    });
  } catch (err) {
    res.status(500).send(err);
```

```
    }
});

// ============================================
// 8. PAYROLL AND HISTORY
// ============================================


// Employee-specific payroll history
app.get('/api/payroll/employee-history/:employeeId', (req, res) => {
  const { employeeId } = req.params;
  const sql = `
    SELECT
      p.id, p.month_year, p.salary_rate, p.total_hours, p.overtime_hours,
      p.overtime_pay, p.transport_allowance, p.meal_allowance, p.housing_allowance,
      p.sss_deduction, p.philhealth_deduction, p.pagibig_deduction, p.loan_deduction,
      p.other_deduction, p.gross_pay, p.tax_deduction, p.net_pay, p.status
    FROM payroll p
    JOIN users u ON p.user_id = u.id
    WHERE u.employee_id = ?
    ORDER BY p.id DESC
    LIMIT 12
  `;
  db.query(sql, [employeeId], (err, results) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(results);
  });
});

app.get('/api/payroll/history', (req, res) => {
    const sql = `
        SELECT
            p.id,
            p.month_year,
            p.salary_rate,
            p.total_hours,
            p.overtime_hours,
            p.overtime_pay,
            p.transport_allowance,
            p.meal_allowance,
            p.housing_allowance,
            p.sss_deduction,
            p.philhealth_deduction,
            p.pagibig_deduction,
```

```
            p.loan_deduction,
            p.other_deduction,
            p.gross_pay,
            p.tax_deduction,
            p.net_pay,
            p.status,
            u.full_name
        FROM payroll p
        JOIN users u ON p.user_id = u.id
        ORDER BY p.id DESC
    `;
    db.query(sql, (err, results) => {
        if (err) return res.status(500).json({ error: err.message });
        res.json(results);
    });
});

// Update monthly salary and work days per month (replaces old rate update)
app.put('/api/payroll/update-employee-salary/:employeeId', (req, res) => {
    const { employeeId } = req.params;
    const { monthly_salary, work_days_per_month } = req.body;
    const sql = "UPDATE users SET monthly_salary = ?, work_days_per_month = ? WHERE
employee_id = ?";
    db.query(sql, [monthly_salary, work_days_per_month, employeeId], (err, result) => {
        if (err) return res.status(500).json({ success: false, error: err.message });
        if (result.affectedRows === 0) return res.status(404).json({ success: false, message:
"Employee not found" });
        res.json({ success: true });
    });
});

app.post('/api/payroll/finalize', (req, res) => {
    const {
        user_id,
        month_year,
        salary_rate,
        total_hours,
        overtime_hours,
        overtime_pay,
        transport_allowance,
        meal_allowance,
        housing_allowance,
        sss_deduction,
        philhealth_deduction,
```

```
        pagibig_deduction,
        loan_deduction,
        other_deduction,
        gross_pay,
        tax_deduction,
        net_pay,
        total_earnings,
        status
    } = req.body;

    if (!user_id || !month_year || total_hours === undefined || gross_pay === undefined || net_pay
=== undefined) {
        return res.status(400).json({
            success: false,
            error: "Missing required fields: user_id, month_year, total_hours, gross_pay, or net_pay"
        });
    }

    let finalSalaryRate = salary_rate;
    if (!finalSalaryRate && total_hours > 0) {
        finalSalaryRate = gross_pay / total_hours;
    } else if (!finalSalaryRate) {
        finalSalaryRate = 0;
    }

    const checkSql = "SELECT id FROM payroll WHERE user_id = ? AND month_year = ?";
    db.query(checkSql, [user_id, month_year], (err, rows) => {
        if (err) return res.status(500).json({ success: false, error: "Database error during duplicate
check" });
        if (rows.length > 0) return res.status(400).json({ success: false, error: "Payroll already
finalized for this month." });

        const sql = `
            INSERT INTO payroll
            (user_id, month_year, salary_rate, total_hours, overtime_hours, overtime_pay,
             transport_allowance, meal_allowance, housing_allowance,
             sss_deduction, philhealth_deduction, pagibig_deduction, loan_deduction,
other_deduction,
             gross_pay, tax_deduction, net_pay, total_earnings, status)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        `;
        db.query(sql, [
            user_id,
            month_year,
```

```
            finalSalaryRate,
            total_hours,
            overtime_hours || 0,
            overtime_pay || 0,
            transport_allowance || 0,
            meal_allowance || 0,
            housing_allowance || 0,
            sss_deduction || 0,
            philhealth_deduction || 0,
            pagibig_deduction || 0,
            loan_deduction || 0,
            other_deduction || 0,
            gross_pay,
            tax_deduction || 0,
            net_pay,
            total_earnings || net_pay,
            status || 'paid'
        ], (err, result) => {
            if (err) {
                console.error("Payroll finalize error:", err);
                return res.status(500).json({ success: false, error: err.message });
            }
            res.json({ success: true, message: "Payroll record saved successfully!" });
        });
    });
});

app.post('/api/payroll/run-monthly', async (req, res) => {
  const { month, year } = req.body;
  if (!month || !year) return res.status(400).json({ error: "Month and year required" });

  const startDate = `${year}-${String(month).padStart(2, '0')}-01`;
  const endDate = `${year}-${String(month).padStart(2, '0')}-31`;

  try {
    const [employees] = await db.promise().query(
      "SELECT id, employee_id, full_name, monthly_salary, work_days_per_month FROM users
WHERE LOWER(role) = 'instructor' AND status = 'active'"
    );

    const processed = [];
    const skipped = [];

    for (const emp of employees) {
```

```
    const monthYear = new Date(year, month - 1).toLocaleString('default', { month: 'long', year:
'numeric' });

    const [existing] = await db.promise().query(
      "SELECT id FROM payroll WHERE user_id = ? AND month_year = ?",
      [emp.id, monthYear]
    );
    if (existing.length > 0) {
      skipped.push({ employee: emp.full_name, reason: "Already finalized" });
      continue;
    }

    const [attendance] = await db.promise().query(
      `SELECT COALESCE(SUM(TIMESTAMPDIFF(MINUTE, time_in, time_out) / 60), 0) as
total_hours
       FROM attendance
       WHERE user_id = ? AND date BETWEEN ? AND ? AND status IN ('present', 'late')`,
      [emp.employee_id, startDate, endDate]
    );
    const totalHours = attendance[0].total_hours;

    const hourlyRate = (emp.monthly_salary / emp.work_days_per_month) / 8;
    const grossPay = totalHours * hourlyRate;
    const tax = grossPay * 0.10;
    const netPay = grossPay - tax;

    await db.promise().query(
      `INSERT INTO payroll
        (user_id, month_year, salary_rate, total_hours,
         overtime_hours, overtime_pay,
         transport_allowance, meal_allowance, housing_allowance,
         sss_deduction, philhealth_deduction, pagibig_deduction, loan_deduction,
other_deduction,
         gross_pay, tax_deduction, net_pay, status, total_earnings)
       VALUES (?, ?, ?, ?, 0, 0, 0, 0, 0, 0, 0, 0, 0, ?, ?, ?, 'paid', ?)`,
      [emp.id, monthYear, hourlyRate, totalHours, grossPay, tax, netPay, netPay]
    );
    processed.push({ employee: emp.full_name, totalHours, grossPay, netPay });
  }

  res.json({
    success: true,
    processed: processed.length,
    skipped: skipped.length,
```

```
      details: { processed, skipped }
    });
  } catch (err) {
    console.error("Monthly payroll error:", err);
    res.status(500).json({ error: err.message });
  }
});

app.post('/api/payroll/unlock', (req, res) => {
  const { email, pin } = req.body;
  if (!email || !pin) {
    return res.status(400).json({ success: false, message: 'Email and PIN required.' });
  }

  if (!checkPinRateLimit(email)) {
    return res.status(429).json({
      success: false,
      message: 'Too many failed attempts. Please wait 15 minutes before trying again.'
    });
  }

  db.query(
    "SELECT * FROM users WHERE email = ? AND status = 'active' AND role = 'admin'",
    [email],
    (err, results) => {
      if (err) return res.status(500).json({ success: false, message: err.message });
      if (results.length === 0) {
        return res.status(400).json({ success: false, message: 'Admin account not found.' });
      }

      const admin = results[0];
      if (admin.payroll_pin !== pin) {
        recordFailedPinAttempt(email);
        return res.status(400).json({ success: false, message: 'Incorrect security code.' });
      }

      clearPinAttempts(email);

      const payrollToken = jwt.sign(
        { id: admin.id, email: admin.email, purpose: 'payroll-access' },
        payrollJWTSecret,
        { expiresIn: '15m' }
      );
```

```
      // Log access in database
      db.query(
        "INSERT INTO payroll_access_logs (user_id, email) VALUES (?, ?)",
        [admin.id, admin.email],
        (err) => {
          if (err) console.error('Failed to log access:', err);
        }
      );

      res.json({ success: true, token: payrollToken, expiresIn: 900 });
    }
  );
});

// --- Change PIN ---
app.put('/api/users/update-pin', (req, res) => {
  const { email, currentPin, newPin } = req.body;
  if (!email || !currentPin || !newPin) {
    return res.status(400).json({ success: false, message: 'All fields required.' });
  }
  if (newPin.length < 4 || newPin.length > 6 || !/^\d+$/.test(newPin)) {
    return res.status(400).json({ success: false, message: 'PIN must be 4-6 digits.' });
  }

  db.query(
    "SELECT * FROM users WHERE email = ? AND status = 'active' AND role = 'admin'",
    [email],
    (err, results) => {
      if (err) return res.status(500).json({ success: false, message: err.message });
      if (results.length === 0) return res.status(400).json({ success: false, message: 'Admin not
found.' });

      const admin = results[0];
      if (admin.payroll_pin !== currentPin) {
        return res.status(400).json({ success: false, message: 'Current PIN is incorrect.' });
      }

      db.query("UPDATE users SET payroll_pin = ? WHERE id = ?", [newPin, admin.id], (err) => {
        if (err) return res.status(500).json({ success: false, message: 'Failed to update PIN.' });
        clearPinAttempts(email);
        res.json({ success: true, message: 'PIN updated successfully.' });
      });
    }
  );
```

```
});

// --- Payroll access logs ---
app.get('/api/payroll/access-logs', (req, res) => {
  const sql = `
    SELECT pal.email, u.full_name, pal.accessed_at
    FROM payroll_access_logs pal
    JOIN users u ON pal.user_id = u.id
    ORDER BY pal.accessed_at DESC
    LIMIT 100
  `;
  db.query(sql, (err, results) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(results);
  });
});




// Example Node.js Express Route
app.get('/api/reports/analytics', async (req, res) => {
    const { type, month } = req.query;
    try {
        if (type === 'attendance') {
            const [stats] = await db.execute(`
                SELECT
                    AVG(status = 'present') * 100 as avg_attendance,
                    COUNT(CASE WHEN status = 'late' THEN 1 END) as late_arrivals,
                    COUNT(CASE WHEN status = 'absent' THEN 1 END) as absent_days
                FROM attendance
                WHERE DATE_FORMAT(date, '%Y-%m') = ?`, [month]);
            const [chart] = await db.execute(`
                SELECT DATE_FORMAT(date, '%d') as name, COUNT(*) as total
                FROM attendance
                WHERE DATE_FORMAT(date, '%Y-%m') = ? AND status = 'present'
                GROUP BY name`, [month]);
            res.json({
                stats: [
                    { label: 'Avg. Attendance', value: `${Math.round(stats[0].avg_attendance)}%` },
                    { label: 'Late Arrivals', value: stats[0].late_arrivals },
                    { label: 'Absent Days', value: stats[0].absent_days }
                ],
                chart: chart
```

```
            });
        }
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

app.get('/api/employees/last-id', (req, res) => {
  db.query("SELECT employee_id FROM users WHERE employee_id REGEXP '^E[0-9]+$'
ORDER BY CAST(SUBSTRING(employee_id,2) AS UNSIGNED) DESC LIMIT 1", (err, result)
=> {
    if (err) return res.status(500).json({ error: err.message });
    const lastId = result[0]?.employee_id || 'E000';
    res.json({ lastId });
  });
});

// Edit attendance record
app.put('/api/attendance/:id', (req, res) => {
  const { time_in, time_out, status, location } = req.body;
  const attendanceId = req.params.id;
  const sql = `
    UPDATE attendance
    SET time_in = ?, time_out = ?, status = ?, location = ?,
        total_hours = ROUND(TIMESTAMPDIFF(MINUTE, time_in, time_out) / 60, 2)
    WHERE id = ?
  `;
  db.query(sql, [time_in, time_out, status, location, attendanceId], (err) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json({ success: true });
  });
});

app.get('/api/attendance-monthly', async (req, res) => {
  const { month, year } = req.query;
  if (!month || !year) return res.status(400).json({ error: "Month and year required" });

  const startDate = `${year}-${String(month).padStart(2, '0')}-01`;
  const endDate = `${year}-${String(month).padStart(2, '0')}-31`;

  try {
    // Get all active instructors
    const [employees] = await db.promise().query(
```

```
    "SELECT employee_id, full_name FROM users WHERE LOWER(role) = 'instructor' AND
status = 'active'"
   );

   const results = [];

   for (const emp of employees) {
    // Regular hours: attendance records
    const [attendance] = await db.promise().query(
     `SELECT COALESCE(SUM(TIMESTAMPDIFF(MINUTE, time_in, time_out) / 60), 0) as
total_hours
      FROM attendance
      WHERE user_id = ? AND date BETWEEN ? AND ? AND status IN ('present', 'late')`,
     [emp.employee_id, startDate, endDate]
    );
    const regularHours = attendance[0].total_hours;

    // Overtime: for simplicity, we'll compute overtime from schedules (assume any hours worked
beyond schedule end_time)
    // This requires schedule data. For now, return 0; you can enhance later.
    const overtimeHours = 0; // Placeholder

    // Leave days
    const [leaves] = await db.promise().query(
     "SELECT COUNT(*) as leave_days FROM leave_requests WHERE user_id = ? AND
status = 'Approved' AND request_date BETWEEN ? AND ?",
     [emp.employee_id, startDate, endDate]
    );
    const leaveDays = leaves[0].leave_days;

    results.push({
     employee_id: emp.employee_id,
     full_name: emp.full_name,
     regular_hours: regularHours,
     overtime_hours: overtimeHours,
     leave_days: leaveDays
    });
   }

   res.json(results);
  } catch (err) {
   console.error("Attendance monthly error:", err);
   res.status(500).json({ error: err.message });
  }
```

```
});

// GET visitor history (last 200 records)
app.get('/api/visitor-history', (req, res) => {
  const sql = `
    SELECT vh.*, u.full_name as visitor_name
    FROM visitor_history vh
    LEFT JOIN users u ON vh.visitor_id = u.employee_id
    ORDER BY vh.timestamp DESC
    LIMIT 200
  `;
  db.query(sql, (err, results) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(results);
  });
});

// Optional: POST endpoint to log a visitor event (called when BLE position updates)
app.post('/api/visitor-history/log', (req, res) => {
  const { visitor_id, visitor_name, ble_id, floor, x, y, event_type } = req.body;
  const sql = `INSERT INTO visitor_history (visitor_id, visitor_name, ble_id, floor, x, y,
event_type) VALUES (?, ?, ?, ?, ?, ?, ?)`;
  db.query(sql, [visitor_id, visitor_name, ble_id, floor, x, y, event_type || 'move'], (err) => {
    if (err) console.error(err);
    res.json({ success: true });
  });
});

// ============================================

// 9. SCHOOL LOCATIONS & COURSES MANAGEMENT
// ============================================


app.get('/api/school-locations', (req, res) => {
  db.query("SELECT * FROM school_locations ORDER BY name", (err, result) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(result);
  });
});

app.post('/api/school-locations', (req, res) => {
  const { name, latitude, longitude, radius } = req.body;
  if (!name) return res.status(400).json({ error: "Location name required" });
```

```
  const sql = "INSERT INTO school_locations (name, latitude, longitude, radius) VALUES (?, ?,
?, ?)";
  db.query(sql, [name, latitude || 0, longitude || 0, radius || 200], (err, result) => {
    if (err) {
      if (err.code === 'ER_DUP_ENTRY') return res.status(400).json({ error: "Location already
exists" });
      return res.status(500).json({ error: err.message });
    }
    res.json({ success: true, id: result.insertId });
  });
});

app.put('/api/school-locations/:id', (req, res) => {
  const { name, latitude, longitude, radius } = req.body;
  const sql = "UPDATE school_locations SET name=?, latitude=?, longitude=?, radius=?
WHERE id=?";
  db.query(sql, [name, latitude, longitude, radius, req.params.id], (err, result) => {
    if (err) return res.status(500).json({ error: err.message });
    if (result.affectedRows === 0) return res.status(404).json({ error: "Location not found" });
    res.json({ success: true });
  });
});

app.delete('/api/school-locations/:id', (req, res) => {
  db.query("DELETE FROM school_locations WHERE id = ?", [req.params.id], (err, result) => {
    if (err) return res.status(500).json({ error: err.message });
    if (result.affectedRows === 0) return res.status(404).json({ error: "Location not found" });
    res.json({ success: true });
  });
});

// ----- Courses -----
app.get('/api/courses', (req, res) => {
  db.query("SELECT * FROM courses ORDER BY name", (err, result) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(result);
  });
});

app.post('/api/courses', (req, res) => {
  const { name } = req.body;
  if (!name) return res.status(400).json({ error: "Course name required" });
  const sql = "INSERT INTO courses (name) VALUES (?)";
  db.query(sql, [name], (err, result) => {
```

```
    if (err) {
      if (err.code === 'ER_DUP_ENTRY') return res.status(400).json({ error: "Course already
exists" });
      return res.status(500).json({ error: err.message });
    }
    res.json({ success: true, id: result.insertId });
  });
});

app.put('/api/courses/:id', (req, res) => {
  const { name } = req.body;
  const sql = "UPDATE courses SET name=? WHERE id=?";
  db.query(sql, [name, req.params.id], (err, result) => {
    if (err) return res.status(500).json({ error: err.message });
    if (result.affectedRows === 0) return res.status(404).json({ error: "Course not found" });
    res.json({ success: true });
  });
});

app.delete('/api/courses/:id', (req, res) => {
  db.query("DELETE FROM courses WHERE id = ?", [req.params.id], (err, result) => {
    if (err) return res.status(500).json({ error: err.message });
    if (result.affectedRows === 0) return res.status(404).json({ error: "Course not found" });
    res.json({ success: true });
  });
});

// Conflict check helper: returns busy intervals for an employee on a given date
app.get('/api/schedules/conflicts', (req, res) => {
  const { user_id, date, exclude_id } = req.query;
  if (!user_id || !date) return res.status(400).json({ error: "user_id and date required" });
  let sql = "SELECT start_time, end_time FROM schedules WHERE user_id = ? AND date = ?";
  const params = [user_id, date];
  if (exclude_id) {
    sql += " AND id != ?";
    params.push(exclude_id);
  }
  sql += " ORDER BY start_time";
  db.query(sql, params, (err, results) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(results); // array of {start_time, end_time}
  });
});
```

```
// OTP LOGIN
// ==========================================
const otpStore = {}; // temporary in-memory OTP storage

// Generate and send OTP
app.post('/api/auth/send-otp', (req, res) => {
  const { email } = req.body;
  if (!email) return res.status(400).json({ success: false, message: 'Email required' });

  // Check if user exists
  db.query("SELECT * FROM users WHERE email = ? AND status = 'active'", [email], (err,
results) => {
    if (err) return res.status(500).json({ success: false, message: err.message });
    if (results.length === 0) return res.status(404).json({ success: false, message: 'No active
account found with that email.' });

    // Generate 6 digit OTP
    const otp = Math.floor(100000 + Math.random() * 900000).toString();
    const expiresAt = Date.now() + 5 * 60 * 1000; // expires in 5 minutes

    otpStore[email] = { otp, expiresAt };
    console.log(`[OTP] ${email} → ${otp}`);

    // Send email
    const mailOptions = {
      from: 'yapkimjean@gmail.com',
      to: email,
      subject: 'Your OTP Code - UniVITA',
      text: `Your OTP code is: ${otp}\n\nThis code expires in 5 minutes.\n\nIf you did not request
this, please ignore this email.`
    };

    transporter.sendMail(mailOptions, (error) => {
      if (error) {
        console.error('OTP email error:', error);
        return res.status(500).json({ success: false, message: 'Failed to send OTP email.' });
      }
      res.json({ success: true, message: 'OTP sent to your email.' });
    });
  });
```

```
});

// Verify OTP
app.post('/api/auth/verify-otp', (req, res) => {
  const { email, otp } = req.body;
  if (!email || !otp) return res.status(400).json({ success: false, message: 'Email and OTP
required' });

  const record = otpStore[email];

  if (!record) return res.status(400).json({ success: false, message: 'No OTP found. Please
request a new one.' });
  if (Date.now() > record.expiresAt) {
    delete otpStore[email];
    return res.status(400).json({ success: false, message: 'OTP has expired. Please request a
new one.' });
  }
  if (record.otp !== otp) return res.status(400).json({ success: false, message: 'Invalid OTP.' });

  // OTP is correct — clean up and return user
  delete otpStore[email];

  db.query("SELECT * FROM users WHERE email = ? AND status = 'active'", [email], (err,
results) => {
    if (err) return res.status(500).json({ success: false, message: err.message });
    if (results.length === 0) return res.status(404).json({ success: false, message: 'User not
found.' });

    res.json({ success: true, user: results[0] });
  });
});


const PORT = 5000;
app.listen(PORT, '0.0.0.0', () => {
    console.log(`UniVITA Backend running on http://0.0.0.0:${PORT}`);
});
```